

# Giải tích số: Sử dụng lặp Jacobi tìm ma trận nghịch đảo

## 1 Cơ sở lý thuyết

### 1.1 1. Đưa về bài toán giải hệ phương trình tuyến tính

Ma trận nghịch đảo  $X = A^{-1}$  của ma trận hệ số  $A$  cấp  $n$  phải thỏa mãn phương trình ma trận:

$$AX = I \quad (1)$$

Trong đó,  $I$  là ma trận đơn vị cấp  $n$ . Về bản chất toán học, việc tìm ma trận  $X$  tương đương với việc giải đồng thời  $n$  hệ phương trình tuyến tính độc lập có cùng cấu trúc ma trận hệ số  $A$ . Vế phải của từng hệ phương trình tương ứng là các vector cột của ma trận đơn vị  $I$ .

### 1.2 2. Xây dựng công thức lặp Jacobi dạng ma trận

Tiến hành phân rã ma trận hệ số  $A$  thành tổng của hai ma trận thành phần:  $A = D + R$ , trong đó:

- $D$ : Ma trận đường chéo chứa các phần tử  $a_{ii}$  nằm trên đường chéo chính của  $A$ .
- $R$ : Ma trận phần dư chứa các phần tử nằm ngoài đường chéo chính của  $A$  ( $r_{ii} = 0, r_{ij} = a_{ij}$  với  $\forall i \neq j$ ).

Thế phép phân rã vào phương trình ban đầu, ta có:

$$(D + R)X = I \implies DX = I - RX \implies X = D^{-1}(I - RX) \quad (2)$$

Do  $D$  là ma trận đường chéo, phép tính nghịch đảo  $D^{-1}$  được thực hiện triệt để với chi phí tính toán cực thấp (nghịch đảo từng phần tử trên đường chéo:  $d_{ii}^{-1} = \frac{1}{a_{ii}}$ ). Từ hệ thức giải tích trên, ta thiết lập chuỗi lặp Jacobi tổng quát trên không gian ma trận:

$$X^{(k+1)} = D^{-1}(I - RX^{(k)}), \quad k = 0, 1, 2, \dots \quad (3)$$

### 1.3 3. Điều kiện hội tụ và Hệ số co

Dãy ma trận lặp  $X^{(k)}$  sẽ hội tụ về nghiệm chính xác  $A^{-1}$  với mọi ma trận khởi tạo  $X^{(0)}$  khi và chỉ khi bán kính phổ (spectral radius)  $\rho$  của ma trận lặp  $T = -D^{-1}R$  thỏa mãn điều kiện:

$$\rho(D^{-1}R) < 1 \quad (4)$$

**Điều kiện đủ thực hành:** Tương tự bài toán hệ phương trình, phương pháp lặp Jacobi ma trận được bảo đảm hội tụ tuyệt đối nếu  $A$  là ma trận **chéo trội nghiêm ngặt** (strictly diagonally dominant) theo hàng hoặc theo cột.

## 1.4 4. Tối ưu hóa ma trận khởi tạo $X^{(0)}$ và Độ phức tạp

- **Khởi tạo:** Lựa chọn kinh điển là  $X^{(0)} = \mathbf{0}$  (ma trận không). Tuy nhiên, để tối ưu hóa số vòng lặp, ta thường khởi tạo  $X^{(0)} = D^{-1}$ . Điều này tương đương với việc triệt tiêu được chu kỳ tính toán của bước lặp đầu tiên.
- **Độ phức tạp thuật toán (Computational Complexity):** Tại mỗi bước lặp, hệ thống phải thực hiện phép nhân ma trận  $R \times X^{(k)}$ . Do đó, độ phức tạp thời gian cho mỗi chu kỳ lặp là  $\mathcal{O}(n^3)$ . Việc sử dụng các thuật toán nhân ma trận song song có thể tối ưu hóa quy trình này trên thực tế.

## 2 Ví dụ minh họa tính toán chi tiết

Tìm ma trận nghịch đảo của  $A = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix}$ . Nhận thấy  $|4| > |1|$ , ma trận  $A$  chéo trội nghiêm ngặt, đảm bảo tính hội tụ.

**Phân rã ma trận:**

$$D = \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix} \implies D^{-1} = \begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}$$
$$R = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Thiết lập ma trận khởi tạo tối ưu  $X^{(0)} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$ .

**Chu kỳ lặp 1 ( $k = 0$ ):**

$$X^{(1)} = D^{-1} (I - RX^{(0)}) = D^{-1}I = D^{-1} = \begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}$$

**Chu kỳ lặp 2 ( $k = 1$ ):**

$$RX^{(1)} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix} = \begin{bmatrix} 0 & 0.25 \\ 0.25 & 0 \end{bmatrix}$$
$$I - RX^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \begin{bmatrix} 0 & 0.25 \\ 0.25 & 0 \end{bmatrix} = \begin{bmatrix} 1 & -0.25 \\ -0.25 & 1 \end{bmatrix}$$
$$X^{(2)} = D^{-1}(I - RX^{(1)}) = \begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix} \begin{bmatrix} 1 & -0.25 \\ -0.25 & 1 \end{bmatrix} = \begin{bmatrix} 0.25 & -0.0625 \\ -0.0625 & 0.25 \end{bmatrix}$$

Khảo sát nghiệm giải tích chính xác:  $A^{-1} = \frac{1}{15} \begin{bmatrix} 4 & -1 \\ -1 & 4 \end{bmatrix} \approx \begin{bmatrix} 0.2667 & -0.0667 \\ -0.0667 & 0.2667 \end{bmatrix}$ . Kết quả thực nghiệm tại  $X^{(2)}$  cho thấy biên độ sai số đã hội tụ rất nhanh về ngưỡng ổn định.

### 3 Thuật toán và Mã giả (Pseudocode)

Cấu trúc logic của phương pháp lặp Jacobi dạng ma trận được mô hình hóa thành đoạn mã giả dưới đây. Thuật toán sử dụng chuẩn ma trận vô cùng (infinity norm) nhằm đánh giá sai số hội tụ.

---

**Algorithm 1** Phương pháp Lặp Jacobi tìm nghịch đảo ma trận

---

**Require:** Ma trận vuông  $A$  cấp  $n \times n$  (thỏa mãn chéo trội), dung sai  $\epsilon$ , giới hạn vòng lặp  $N_{max}$

**Ensure:** Ma trận nghịch đảo  $X \approx A^{-1}$  hoặc thông báo lỗi phân kỳ

```
1: Khởi tạo ma trận đơn vị  $I$  cấp  $n \times n$ 
2: Phân rã  $A$  thành ma trận đường chéo  $D$  và ma trận phần dư  $R$  ( $A = D + R$ )
3: Khởi tạo ma trận  $D_{inv}$  cấp  $n \times n$ 
4: for  $i = 1, 2, \dots, n$  do ▷ Tính ma trận nghịch đảo của  $D$ 
5:   if  $A_{ii} = 0$  then
6:     return "Lỗi: Phần tử trên đường chéo chính bằng 0."
7:   end if
8:    $D_{inv}[i][i] \leftarrow \frac{1}{A_{ii}}$ 
9: end for
10:  $X \leftarrow D_{inv}$  ▷ Khởi tạo  $X^{(0)} = D^{-1}$  để tiết kiệm 1 chu kỳ lặp
11:  $k \leftarrow 1$ 
12: while  $k \leq N_{max}$  do
13:    $P \leftarrow R \times X$  ▷ Thực hiện phép nhân ma trận ( $n \times n$ )
14:    $M \leftarrow I - P$  ▷ Tính ma trận trung gian
15:    $X_{new} \leftarrow D_{inv} \times M$  ▷ Cập nhật ma trận xấp xỉ mới
16:    $error \leftarrow \|X_{new} - X\|_{\infty}$  ▷ Tính sai số theo chuẩn vô cùng của ma trận
17:   if  $error < \epsilon$  then ▷ Kiểm tra tiêu chí dừng
18:     return  $X_{new}$ 
19:   end if
20:    $X \leftarrow X_{new}$  ▷ Đồng bộ hóa không gian bộ nhớ
21:    $k \leftarrow k + 1$ 
22: end while
23: return "Thuật toán không đạt dung sai yêu cầu sau  $N_{max}$  vòng lặp. Kết quả tạm thời:  $X$ ."
```

---